Project group 12 - Semantic Segmentation

Authors:

Nathan Challande 117110 nathan.challande@epfl.ch

Vasco Oliveira 106561 vasco.noites@tecnico.ulisboa.pt

Gustavo Monteiro da Rosa 114876 gustavo.monteiro.rosa@tecnico.ulisboa.pt

Index

1. Introduction 1
2. Network Architecture: DeepLabV3+ and ResNet-18 2
3. Methodology 2
   3.1 Computational Environment and Model Acquisition 2
   3.2 Baseline Evaluation of the Pre-Trained Model 3
   3.3 Transfer Learning Pipeline design 12
   3.4 Network Surgery 14
   3.5 Model Training 15
4. Results and Discussion 16
   4.1 Model Accuracy 16
   4.2 Evaluation on Validation Images 18
   4.3 Validation For Filtered Classes 21
   4.4 Inspection Of One Image 23
5. Conclusions 25
6. References 26
Appendix. Functions used 26

1. Introduction

Robust perception is a fundamental requirement for autonomous driving systems, as they must accurately interpret complex and dynamic environments in real time. In particular, a deep learning model must reliably recognize critical classes such as vehicles, bicycles, pedestrians, and road surfaces in order to remain on track and safely avoid obstacles. Semantic segmentation addresses this challenge by assigning a categorical label (associated with a color map) to every pixel in an image, enabling a detailed understanding of drivable areas, obstacles, and key navigational elements.

The objective of this project is to develop a semantic segmentation pipeline capable of identifying the most relevant environmental classes in road scenes (e.g., Asphalt, Vehicles, People, background). To achieve this, a transfer learning approach was adopted, leveraging a pre-trained DeepLabV3+ with a ResNet-18 backbone, originally trained on the 11-class CamVid_v2 dataset. The model was then adapted to a custom dataset (FCCD), which contains images of coastal road environments from the Floripa region. The training set consists

of 400 images, while the validation set includes 100 images. All images were uniformly resized to match the network's input dimensions, ensuring compatibility during inference. Additionally, the output layer of the pre-trained model was modified to support 17 classes, aligning it with the labeling scheme of the FCCD dataset.

2. Network Architecture: DeepLabV3+ and ResNet-18

The core of our segmentation pipeline relies on the DeepLabV3+ architecture with a ResNet-18 backbone. ResNet-18 serves as the feature extractor, designed specifically to mitigate the vanishing gradient problem found in deep networks through the use of residual connections.

Instead of forcing the network layers to learn an entirely new, unreferenced mapping, these layers learn a residual function. In simple terms:

$$\text{Output} = F(\text{Input}, \text{Weights}) + \text{Input}$$

By passing the original input directly through to the output, the network only has to learn the *difference* (the residual) needed to reach the desired feature map, making optimization significantly more stable.

DeepLabV3+ enhances this by employing Atrous Spatial Pyramid Pooling (ASPP). ASPP applies dilated convolutions (technique that inserts gaps between kernel elements) at multiple sampling rates, enlarging the network's field of view to capture multi-scale contextual information without increasing the computational load.

3. Methodology

3.1 Computational Environment and Model Acquisition

To ensure deterministic execution and prevent variable shadowing, the pipeline initializes by flushing workspace memory (`clear`), clearing the console (`clc`), and terminating existing graphic handles (`close all`). A global figure handle is allocated to standardize the targeting of visual outputs.

For model acquisition, the script uses a cross-platform retrieval mechanism (`fullfile`, `tempdir`) to dynamically map to the OS's temporary directory. It employs defensive programming (`~exist`) to check the local cache before initiating a REST GET request (`websave`) for the 58 MB DeepLabV3+ weights. This caching strategy prevents redundant downloads and accelerates iterative testing. Finally, the asset is unpacked, loading the pre-trained Directed Acyclic Graph (DAG) into memory for subsequent network surgery.

```
clear, clc, close all

report_viewer = false; %in order to export to pdf put this to false.

%Import deeplab v3 model
pretrainedURL = "https://ssd.mathworks.com/supportfiles/vision/data/
deeplabv3plusResnet18CamVid_v2.zip";
pretrainedFolder = fullfile(tempdir,"pretrainedNetwork");
pretrainedNetworkZip =
fullfile(pretrainedFolder,"deeplabv3plusResnet18CamVid_v2.zip");
if ~exist(pretrainedNetworkZip,'file')
```

```
    mkdir(pretrainedFolder);
    disp("Downloading pretrained network (58 MB)...");
    websave(pretrainedNetworkZip,pretrainedURL);
end
unzip(pretrainedNetworkZip, pretrainedFolder)
%Load the model to use it for predicition
pretrainedNetwork =
fullfile(pretrainedFolder,"deeplabv3plusResnet18CamVid_v2.mat");
data = load(pretrainedNetwork);
```

```
Warning: While loading an object of class 'deep.TrainingInfo':
Unrecognized field name "MetricData".
```

```
net = data.net;
```

Storing the classes of the pre-trained model to use them and select the most useful for autonomous driving later:

```
classes = [
    "Sky" %C1
    "Building" %C2
    "Pole" %C3
    "Road" %C4
    "Pavement" %C5
    "Tree" %C6
    "SignSymbol" %C7
    "Fence" %C8
    "Car" %C9
    "Pedestrian" %C10
    "Bicyclist" %C11
    ];
```

3.2 Baseline Evaluation of the Pre-Trained Model

Prior to executing network surgery and transfer learning, a baseline evaluation of the unmodified DeepLabV3+ network was conducted using a sample image from the custom dataset. To ensure topological compatibility, the raw input image was systematically resized to match the exact spatial dimensions expected by the ResNet-18 input layer.

A forward pass was then executed using the `semanticseg` function. During this process, the network evaluates the spatial features and outputs a two-dimensional categorical tensor. At the computational level, the model assigns a discrete class identifier (e.g., C1, C2, C3) to every single pixel. For human interpretability, these identifiers map directly to the original 11 CamVid classes (e.g., C1 corresponds to "Sky").

To visually validate the model's zero-shot performance on our specific road scenes, this raw categorical output was mapped to a standardized CamVid colormap and alpha-blended over the original RGB image. This qualitative test confirmed the network's foundational ability to recognize generic features before being fine-tuned to our specific 17-class dataset

```
% Image List
```

```
imageFiles = [
    "train/im/20230803_115346_050.jpg"
    "train/im/20230803_115352_050.jpg"
];

numImgs = numel(imageFiles);

% Title
g12figure("Multiple Images - Original");

for i = 1:numImgs

    % Load image
    I = imread(imageFiles(i));

    % Create subplot
    ax = subplot(1, numImgs, i);
    imshow(I, 'Parent', ax);
    axis(ax, 'off');
    title(ax, '');

    % Manually adjust image size
    if numImgs == 2
        if i == 1
            ax.Position = [0.05 0.12 0.42 0.98];
        else
            ax.Position = [0.53 0.12 0.42 0.98];
        end

    end

end
```

Figure 1 - Multiple Images - Original



```
% Resize info
inputSize = net.Layers(1).InputSize;

% Colormap
cmap = camvidColorMap_group12;

g12figure("Multiple Images - Pretrained Model");

% Store images
tileImgs = cell(1, numImgs);

for i = 1:numImgs

    % Load image
    I = imread(imageFiles(i));

    % Resize
    I = imresize(I, inputSize(1:2));

    % Semantic segmentation
    C = semanticseg(I, net);

    % Overlay
    B = labeloverlay(I, C, ...
        'Colormap', cmap, ...
        'Transparency', 0.4);
```

```
    tileImgs{i} = B;
end

% Create tiled image (1 row, numImgs columns)
bigImage = imtile(tileImgs, ...
    'GridSize', [1 numImgs], ...
    'BorderSize', 15, ...
    'BackgroundColor', [1 1 1]);

% Display main image block
mainAx = axes('Position', [0.05 0.12 0.78 0.75]);
imshow(bigImage, 'Parent', mainAx);
axis(mainAx, 'off');

% Legend axis (separate, no distortion)
legendAx = axes('Position',[0.86 0.2 0.03 0.6]);
axis(legendAx, 'off');
axes(legendAx);
pixelLabelColorbar_group12(cmap, classes);
```

Figure 2 - Multiple Images - Pretrained Model



To align the baseline model's output with our specific project objectives, a targeted subset of three high-priority semantic classes was defined: "Road" (C4), "Tree" (C6), and "Car" (C9).

A custom RGB colormap matrix was established to visually distinguish these specific classes.

To isolate these specific predictions within the image, it was applied a vectorized logical masking to the categorical output tensor (`C`). Instead of computationally expensive pixel-by-pixel iteration, a logical OR operator (`|`) was used to generate a boolean mask (`restrict`). This mask instantly evaluates the entire matrix, returning a logical `1` (true) for any pixel classified as C4, C6, or C9, and a `0` (false) for all others.

```
new_classes = [
     "Road" %C4
     "Tree" %C6
     "Car" %C9
];

new_cmap = [
    128 64 128    % Road
    128 128 0     % Tree
    128 0 0       % Car
    ];

new_cmap = new_cmap ./ 255;

%Before performing the semantic segm. needed to restrict the model to
%output only 3 outputs

new_C = C;

restrict = new_C == "C4" | new_C == "C6" | new_C == "C9";

disp('Categories before removal (new_C):');
```

```
Categories before removal (new_C):
```

```
disp(strjoin(categories(new_C), ', '));
```

```
C1, C2, C3, C4, C5, C6, C7, C8, C9, C10, C11
```

```
new_C(~restrict) = missing; % if not in restrict set to undefined (missing is the
matlab default undefined)
disp('Pixels matching restricted classes:');
```

```
Pixels matching restricted classes:
```

```
disp(sum(restrict(:)));
```

```
      553016
```

```
new_C = removecats(new_C); % remove non-used classes
disp('Categories after removal (new_C):');
```

```
Categories after removal (new_C):
```

```
disp(strjoin(categories(new_C), ', '));
```

```
C4, C6, C9
```

To qualitatively assess the isolated semantic classes, the filtered categorical tensor was superimposed onto the original image using an alpha-blended overlay (`Transparency = 0.4`). To maximize the visual contrast and ensure the semantic labels were easily distinguishable, the base RGB image was temporarily cast to grayscale prior to the overlay. A comparative visualization was then generated, displaying the raw input alongside the segmented output, complete with a custom categorical legend (`pixelLabelColorbar_group12`) for immediate interpretability.

Furthermore, **this entire filtering and visualization sequence was abstracted into a custom, user-defined function named `g12_modelclass_output`.** This modular approach eliminates redundant scripting, significantly enhances the interpretability of the main Live Script, and provides a highly user-friendly interface for rapidly evaluating different class combinations in future testing phases.

```
g12figure("Model output with selected classes");

% Preallocate cell array for all images to tile
tileImgs = cell(numImgs, 2);

% Get network input size once
inputSize = net.Layers(1).InputSize;

for i = 1:numImgs

    % Load image
    I = imread(imageFiles(i));

    % Resize
    I_resized = imresize(I, inputSize(1:2));

    % Semantic segmentation
    C = semanticseg(I_resized, net);

    % Keep only selected classes
    new_C = C;
    restrict = new_C == "C4" | new_C == "C6" | new_C == "C9";
    new_C(~restrict) = missing;
    new_C = removecats(new_C);

    % Overlay
    B = labeloverlay(rgb2gray(I_resized), new_C, ...
        'Colormap', new_cmap, ...
        'Transparency', 0.4);

    % Store images for tiling
    tileImgs{i,1} = I_resized;
    tileImgs{i,2} = B;
end

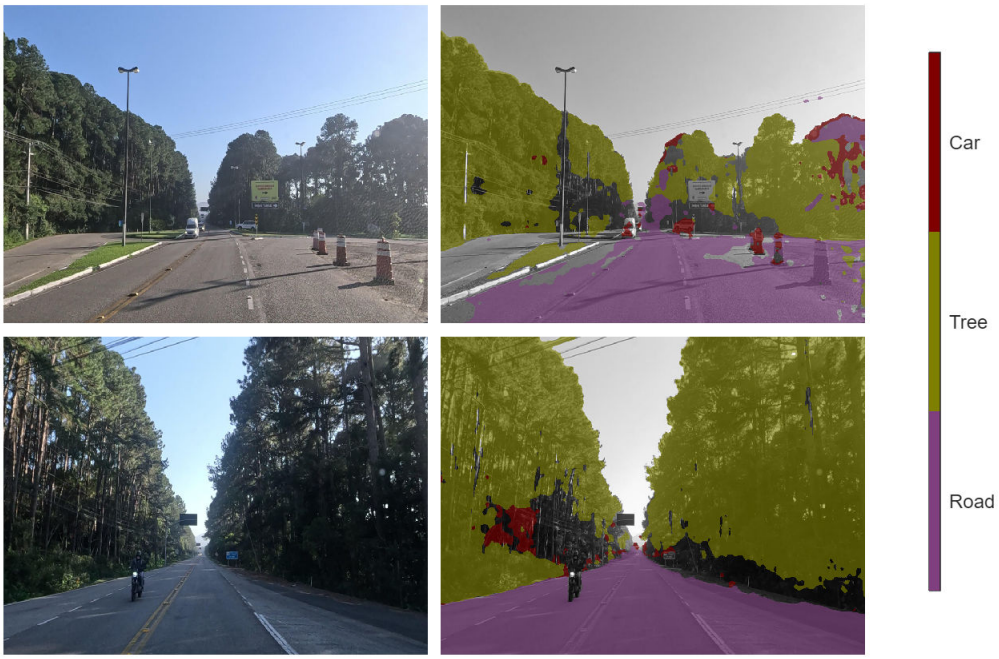
% Convert cell array to vector for imtile
tileImgsLinear = reshape(tileImgs.', 1, []);
```

```
% Create one composite image with 2 columns and numImgs rows
bigImage = imtile(tileImgsLinear, ...
    'GridSize', [numImgs 2], ...
    'BorderSize', 15, ...
    'BackgroundColor', [1 1 1]);

% Main axis for the whole image block
mainAx = axes('Position', [0.05 0.08 0.78 0.82]);
imshow(bigImage, 'Parent', mainAx);
axis(mainAx, 'off');

% Separate axis for the legend
legendAx = axes('Position', [0.86 0.18 0.03 0.64]);
axis(legendAx, 'off');
axes(legendAx);
pixelLabelColorbar_group12(new_cmap, new_classes);
```

Figure 3 - Model output with selected classes




```
drawnow;
```

Next, a new targeted subset of useful classes have been used to assess the quality of the pre-trained model for the autonomous driving task.

```
% List of images
imageFiles = [
    "train/im/20230803_115352_050.jpg"
    "train/im/20230803_115346_050.jpg"
```

```
];

selected_classes = ["Road","Bicyclist","Pedestrian","Car"];

inputSize = net.Layers(1).InputSize;
numImgs = numel(imageFiles);

for i = 1:numImgs

    % Load image
    I = imread(imageFiles(i));

    % Resize
    I = imresize(I, inputSize(1:2));

    % Semantic segmentation
    C = semanticseg(I, net);

    % Filter selected classes
    [new_C, new_cmap, new_classes] = ...
        g12_modelclass_output(I, C, cmap, selected_classes, classes);

    % Overlay
    B = labeloverlay(rgb2gray(I), new_C, ...
        'Colormap', new_cmap, ...
        'Transparency', 0.4);

    % Create one figure per image
    g12figure("Model output to " + length(selected_classes) + " classes");

    % Build one tiled image with the pair
    pairImage = imtile({I, B}, ...
        'GridSize', [1 2], ...
        'BorderSize', 20, ...
        'BackgroundColor', [1 1 1]);

    % Main axis for the image pair
    mainAx = axes('Position', [0.08 0.18 0.68 0.62]);
    imshow(pairImage, 'Parent', mainAx);
    axis(mainAx, 'off');

    % Titles above each image
    annotation('textbox', [0.18 0.79 0.18 0.05], ...
        'String', 'Original Image', ...
        'EdgeColor', 'none', ...
        'HorizontalAlignment', 'center', ...
        'FontWeight', 'bold', ...
        'FontSize', 11);

    annotation('textbox', [0.43 0.79 0.22 0.05], ...
```

```
        'String', 'Segmented Overlay', ...
        'EdgeColor', 'none', ...
        'HorizontalAlignment', 'center', ...
        'FontWeight', 'bold', ...
        'FontSize', 11);

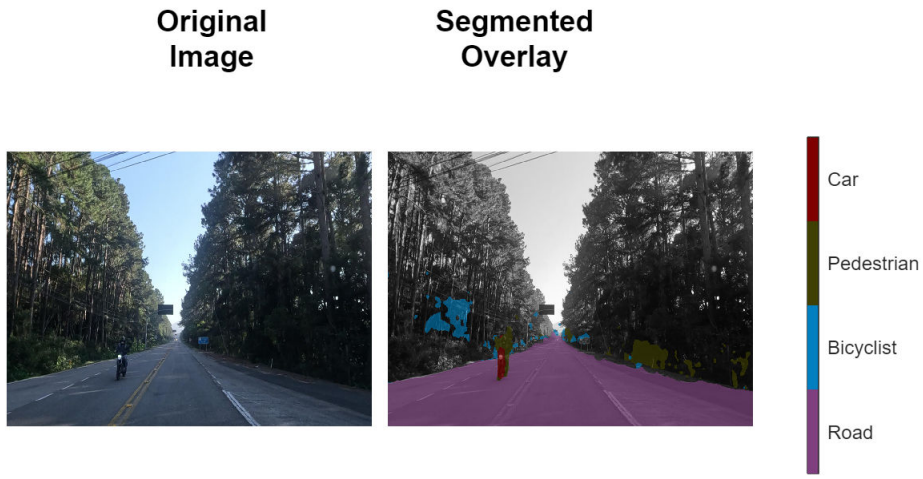
    % Separate legend axis
    legendAx = axes('Position', [0.78 0.27 0.03 0.40]);
    axis(legendAx, 'off');
    axes(legendAx);
    pixelLabelColorbar_group12(new_cmap, new_classes);

    drawnow;
end
```

```
Categories before removal:
C1, C2, C3, C4, C5, C6, C7, C8, C9, C10, C11
Pixels matching restricted classes:
      170416

Categories after removal (still C-codes):
C4, C9, C10, C11
```

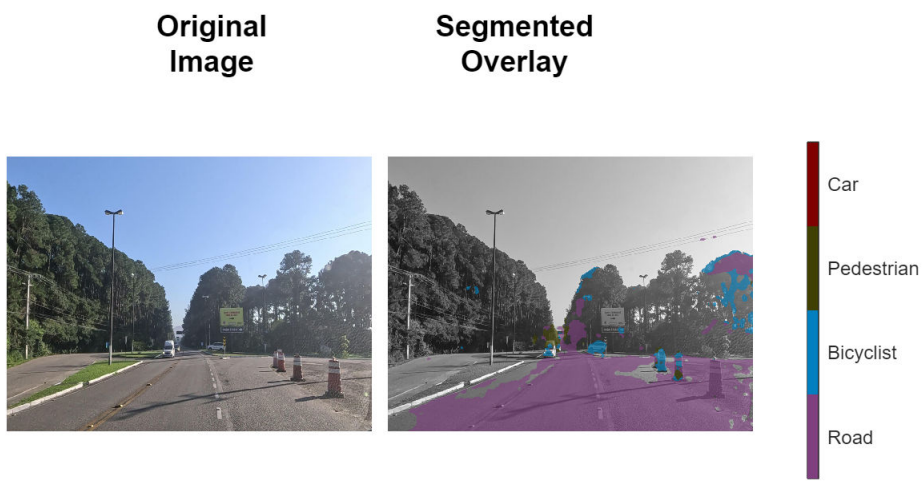
Figure 4 - Model output to 4 classes




```
Categories before removal:
C1, C2, C3, C4, C5, C6, C7, C8, C9, C10, C11
Pixels matching restricted classes:
      161620

Categories after removal (still C-codes):
C4, C9, C10
```

Figure 5 - Model output to 4 classes



The preliminary evaluation of the pre-trained model demonstrates that it is well-suited for autonomous driving applications, as it already achieves strong segmentation performance on the FCCD dataset. The model is able to correctly identify key elements of the road environment, such as the road surface, surrounding structures, and dynamic objects **(cars and bicycles/motorcycles, by visualizing Figure 13 the model actually classified both car and bicicle as the same "entity". Since it is a dinamic object at least it was classified.)**, indicating that it has learned meaningful and transferable visual features. This makes it an excellent starting point for the subsequent transfer learning stage, where the model will be further adapted to the specific characteristics of our dataset.

3.3 Transfer Learning Pipeline design

To begin the transfer learning phase, it was loaded the FCCD dataset, which contains 17 different semantic classes (such as "Asphalt", "Vehicles", and "People"). In the ground-truth mask images, each pixel is assigned an integer value from 0 to 16 to represent its class. It was defined a custom colormap array (`cmap`) to link these raw numbers directly to their correct class names so the network can understand the labels during training.

```
classes = [
    "background"
    "Animals"
    "Asphalt"
    "Cat_s_eye"
    "Cracks"
```

```
    "Dirt_road"
    "Ego"
    "Hard_sand"
    "Markings"
    "Obstacle"
    "People"
    "Pothole"
    "Retaining_wall"
    "Soft_sand"
    "Speed_bump"
    "Vehicles"
    "Wet_sand"
];

cmap = [ %changing the cmap in order to match the excel's dataset
    0
    1
    2
    3
    4
    5
    6
    7
    8
    9
    10
    11
    12
    13
    14
    15
    16
    ];
```

The training and validation sets have been loaded using MATLAB's `imageDatastore` for the raw photographs and `pixelLabelDatastore` for the ground-truth masks.

The training data have been separated from the validation data to ensure the model could be tested on images it had never seen before. Then used the `combine` function to pair each image with its exact corresponding mask.

The pre-trained DeepLabV3+ network requires all input images to be a specific, fixed size. Because the dataset images possessed different dimensions, both the photographs and the corresponding labels required resizing prior to training.

A data transformation was applied utilizing two distinctly different mathematical methods:

1. For the raw images: Standard image resizing (continuous interpolation) was applied to maintain visual smoothness.

2. For the masks (labels): 'Nearest' neighbor resizing was strictly enforced.

```
train_X=imageDatastore('train/im');
train_Y = pixelLabelDatastore('train/gt', classes, cmap);
validation_X = imageDatastore('val/im');
validation_Y = pixelLabelDatastore('val/gt',classes,cmap);
inputSize = net.Layers(1).InputSize;

resizeData = @(data) { ...
    imresize(data{1}, inputSize(1:2)), ...
    imresize(data{2}, inputSize(1:2), 'nearest') ...
};

cds = combine(train_X, train_Y);
cds = transform(cds, resizeData);

val_cds = combine(validation_X, validation_Y);
val_cds = transform(val_cds, resizeData);
```

3.4 Network Surgery

Due to the pre-trained model was configured for the 11-class CamVid dataset, the topology of the Directed Acyclic Graph (DAG) required modification to output 17 classes. A "network surgery" was executed by removing the terminal classification layers (`softmax-out`) and appending a custom sequence of uninitialized layers:

1. **Convolutional Layer (`finalConv`):** A 1x1 convolution layer mapping the high-dimensional feature maps extracted from the `dec_crop2` layer to exactly 17 output channels (one for each target class).
2. **Softmax Layer:** Normalizes the raw output scores of the convolutional layer into a valid probability distribution. The probability that a specific pixel belongs to class "c" is computed as the exponential of the score for class "c", divided by the sum of the exponentials for all 17 classes:
   $$P(\text{class} = c) = \exp(\text{score}_c) / [\exp(\text{score}_1) + \exp(\text{score}_2) + \ldots + \exp(\text{score}_{17})]$$
3. **Pixel Classification Layer:** Computes the categorical Cross-Entropy loss during training. For a true label distribution (Y) and predicted probability (P), the loss for a single pixel is:
   $$\text{Loss} = -[Y_1 * \log(P_1) + Y_2 * \log(P_2) + \ldots + Y_{17} * \log(P_{17})]$$

```
% Define the hyperparameters
opts = trainingOptions('sgdm', ...
    'InitialLearnRate', 1e-3, ...
    'MaxEpochs', 3, ...   % Number of full passes through the training dataset
(Epochs)
    'MiniBatchSize', 4, ...        % Number of samples per training iteration
    'Shuffle', 'every-epoch', ... % Shuffle data at each epoch to improve
generalization
    'Plots', 'training-progress', ... % Display real-time training metrics (loss,
accuracy)
```

```
    'ValidationData', val_cds, ...   % Dataset used to evaluate model during
training
    'ValidationFrequency', 50, ... %perform validation every 50 iterations
    'LearnRateSchedule', 'piecewise', ... % Drop the learning rate as it gets
closer to the minimum
    'LearnRateDropPeriod', 10, ... % Drop learning rate every 10 epochs
    'LearnRateDropFactor', 0.3); % Multiply learning rate by 0.3 at each drop

numClasses = numel(classes); % 17
pxLayer = pixelClassificationLayer('Name','labels');
finalConv = convolution2dLayer(1, numClasses, 'Name', 'finalConv');
softmaxL = softmaxLayer('Name','softmax');

% EXECUTE TRAINING
disp('Initiating Transfer Learning Phase...');
```

```
Initiating Transfer Learning Phase...
```

```
lgraph = layerGraph(net);

% Remove a layer antiga de softmax
lgraph = removeLayers(lgraph, 'softmax-out');

% Adiciona novas layers
lgraph = addLayers(lgraph, finalConv);
lgraph = addLayers(lgraph, softmaxL);
lgraph = addLayers(lgraph, pxLayer);

% Liga tudo
lgraph = connectLayers(lgraph, 'dec_crop2', 'finalConv');
lgraph = connectLayers(lgraph, 'finalConv', 'softmax');
lgraph = connectLayers(lgraph, 'softmax', 'labels');
```

3.5 Model Training

A control mechanism was established to easily switch between training a new model and loading a previously trained one.

When the training mode is activated, the `trainNetwork` function is executed. This process feeds the resized dataset, the modified network layers, and the specified learning parameters into the optimizer to train the network. Once the optimization process concludes, the finished model and its corresponding training metrics are saved directly to a `.mat` file on the disk.

When the training mode is disabled, the script bypasses the time-consuming training step and instead loads the saved model. Before attempting to load the weights into memory, the script performs a basic check to confirm that the file actually exists and contains the necessary network data. This ensures the program does not crash if a user attempts to test the model before it has been trained.

```
training = false;
```

```
if training
    [trainedNet, traininfo] = trainNetwork(cds, lgraph, opts);

    % Guardar corretamente o modelo treinado
    model_filename = 'model_retrained.mat';
    save(model_filename, 'trainedNet', 'traininfo', '-v7.3');
    disp('Model trained and saved successfully.');

else
    model_filename = 'model_retrained.mat';

    if ~isfile(model_filename)
        error("Model file not found: %s", model_filename);
    end

    disp('Loading saved model...');
    saved_data = load(model_filename);

    if ~isfield(saved_data, 'trainedNet')
        error("The file %s does not contain variable 'trainedNet'.",
model_filename);
    end

    trainedNet = saved_data.trainedNet;
    traininfo = saved_data.traininfo;
    disp('Model loaded successfully.');
end
```

```
Loading saved model...
Model loaded successfully.
```

4. Results and Discussion

After training and adapting the neural network through transfer learning, it is essential to evaluate its performance on unseen data. This validation stage aims to assess the model's ability to generalize, identifying both successful predictions and potential misclassifications.

In this context, a testing pipeline was implemented to apply the trained model (`trainedNet`) to a subset of validation images, enabling a qualitative analysis of the semantic segmentation results.

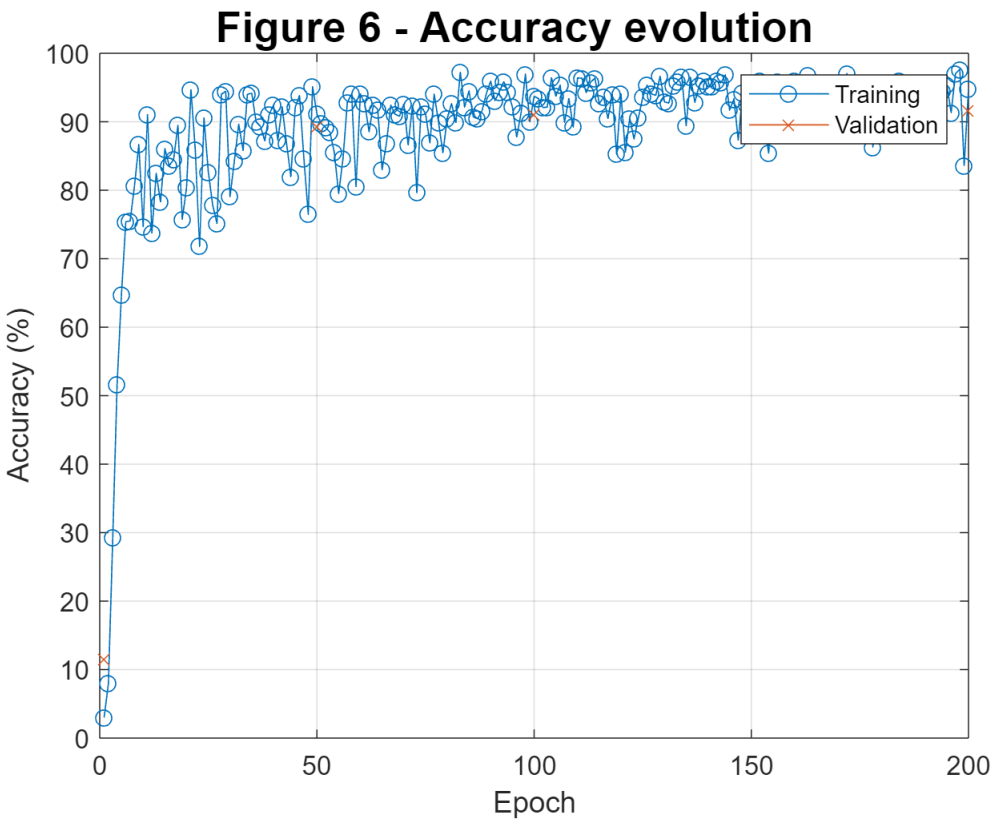
4.1 Model Accuracy

The transfer learning model achieved an overall accuracy of +-90% on the validation dataset (Figure 6), indicating a strong ability to correctly classify the majority of pixels across the images. This result suggests that the pre-trained network was successfully adapted to the new dataset, effectively learning the relevant features required for the target classes. The high accuracy demonstrates that the model is capable of generalizing well, maintaining consistent performance across different validation samples. However, it is important to note that overall accuracy in semantic segmentation can sometimes be influenced by class imbalance, particularly if dominant classes such as background or road occupy a large portion of the images. Therefore, while the 93.3% accuracy (Table 1) reflects a solid performance, it should be interpreted alongside qualitative results and,

ideally, complemented with additional metrics such as class-wise accuracy or Intersection over Union (IoU) for a more comprehensive evaluation.

```
% code to get model accuracy table
epochs = (1:length(traininfo.TrainingAccuracy))';

%creating the accuracy table
if report_viewer
    T = table( ... %Only available on .mlx if report_viewer is true
    epochs, ...
    traininfo.TrainingAccuracy', ...
    traininfo.ValidationAccuracy', ...
    'VariableNames', {'Epoch', 'Training Accuracy (%)', 'Validation Accuracy (%)'}
...
    );
    disp(T)
end
%plotting the table above (accuracy table)
g12figure("Accuracy evolution");
plot(epochs, traininfo.TrainingAccuracy, '-o') %blue line with circles
hold on %in order to join both plots
plot(epochs, traininfo.ValidationAccuracy, '-x') %red X
legend('Training', 'Validation')
%putting labels on each axis
xlabel('Epoch')
ylabel('Accuracy (%)')
hold off
grid on
```




```
drawnow


%printing the final results_
FinalTable = table( ...
    traininfo.FinalValidationAccuracy, ...
    traininfo.FinalValidationLoss, ...
    'VariableNames', {'Final Accuracy (%)', 'Final Loss'} ...
);

disp("Table #1")
```

```
Table #1
```

```
disp(FinalTable)
```

```
    Final Accuracy (%)    Final Loss
    __________________    __________

          93.272           0.25956
```

Validation was executed every 50 epoch, therefore, NaN values appear in the table for epochs where validation was not executed.

4.2 Evaluation on Validation Images

This code performs a qualitative evaluation of a trained semantic segmentation model by applying it to a subset of validation images and visualizing the results. First, it creates an image datastore from the validation folder

and selects 20 images either sequentially (Option A) or randomly (Option B), ensuring flexibility in testing different portions of the dataset. Each image is resized to match the input dimensions required by the trained network, and semantic segmentation is performed using the `semanticseg` function. The resulting categorical label map is then converted into a numerical index representation to ensure compatibility with visualization functions. A dynamic colormap is generated based on the number of predicted classes, and the segmentation is overlaid onto the original image using `labeloverlay`. The outputs are displayed in a 5×4 grid, allowing simultaneous inspection of all processed images. Additionally, error handling is implemented to prevent the execution from stopping if a particular image causes an issue. This process enables a clear and efficient visual assessment of the model's performance across multiple validation samples.

```
%% TEST TRAINED MODEL ON 20 VALIDATION IMAGES
% Works with:
% Option A -> consecutive images starting at startIdx
% Option B -> random images
clearvars -except trainedNet classes

% 1. Create datastore
imdsTest = imageDatastore('val/im');

% 2. Selection mode
selectionMode = 'B';
numImgs = 20;
startIdx = 60;

numAvailable = numel(imdsTest.Files);

if numAvailable == 0
    error('No images were found in val/im');
end

numImgs = min(numImgs, numAvailable);

switch upper(selectionMode)
    case 'A'
        endIdx = min(startIdx + numImgs - 1, numAvailable);
        testFiles = imdsTest.Files(startIdx:endIdx);
        numImgs = numel(testFiles);

    case 'B'
        rng('shuffle');
        idxRand = randperm(numAvailable, numImgs);
        testFiles = imdsTest.Files(idxRand);
end

% 3. Network input size
inputSize = trainedNet.Layers(1).InputSize;

% 4. Colormaps and selected classes
master_cmap = lines(numel(classes));
```

```
master_cmap(1, :) = [0.773 0.686 0.788]; %background as another color
selected_classes = ["Asphalt", "Dirt_road", ...
    "Hard_sand", "Soft_sand", "Vehicles", "People", "Wet_sand"];

new_master_cmap = lines(numel(selected_classes));
new_master_cmap(1, :) = [0.773 0.686 0.788]; %background as another color

%% =========================
%% FULL SEGMENTATION GRID
%% =========================
g12figure("Full Semantic Segmentation Output");

tileImgs_full = cell(1, numImgs);

for k = 1:numImgs
    try
        I = imread(testFiles{k});
        I = imresize(I, inputSize(1:2));

        C = semanticseg(I, trainedNet);

        predClasses = categories(C);
        C_idx = zeros(size(C));

        for i = 1:numel(predClasses)
            C_idx(C == predClasses{i}) = i;
        end

        cmap_dynamic = lines(numel(predClasses));
        cmap_dynamic(1, :) = [0.773 0.686 0.788]; %background as another color

        B = labeloverlay(I, C_idx, ...
            'Colormap', cmap_dynamic, ...
            'Transparency', 0.4);

        tileImgs_full{k} = B;

    catch ME
        warning('Error in image %d: %s', k, ME.message);
        tileImgs_full{k} = uint8(255 * ones(inputSize(1), inputSize(2), 3));
    end
end

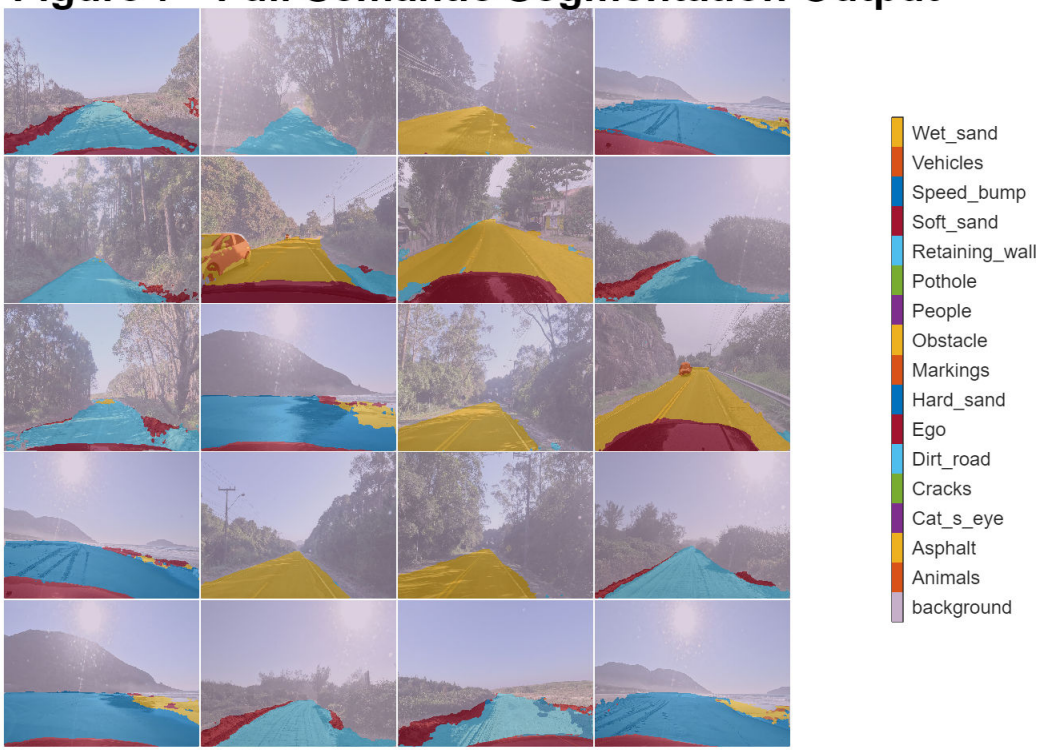
bigImage_full = imtile(tileImgs_full, ...
    'GridSize', [5 4], ...
    'BorderSize', 3, ...
    'BackgroundColor', [1 1 1]);

% Main grid (slightly smaller to make room for legend)
mainAx1 = axes('Position', [0.03 0.05 0.80 0.88]);
```

```
imshow(bigImage_full, 'Parent', mainAx1);
axis(mainAx1, 'off');

% Legend axis
legendAx1 = axes('Position', [0.85 0.20 0.03 0.60]);
axis(legendAx1, 'off');
axes(legendAx1);
pixelLabelColorbar_group12(master_cmap, classes);
```

Figure 7 - Full Semantic Segmentation Output



Note: Figure 7 displays different images from the dataset after each execution of the code.

4.3 Validation For Filtered Classes

This code block generates a visualization of the semantic segmentation results by displaying only the selected classes as colored masks, without overlaying them on the original images. For each of the 20 validation images, the trained model is applied to produce a pixel-wise classification. The resulting categorical segmentation map is then filtered to retain only a predefined subset of relevant classes, while all other classes are discarded. A new RGB image is created with a black background, and pixels belonging to the selected classes are colored according to a predefined colormap. This is achieved by iterating through each selected class, identifying its corresponding pixels, and assigning the appropriate color to those regions. The final masks are displayed in a grid layout, allowing for clear visualization of the spatial distribution of the selected classes without the visual interference of the original image. This representation is particularly useful for analyzing class-specific segmentation performance and isolating relevant features in the scene.

```
%% FILTERED MASKS GRID
%% ========================
```

```
g12figure("Filtered Masks Only - Selected Classes");

tileImgs_mask = cell(1, numImgs);

for k = 1:numImgs
    try
        I = imread(testFiles{k});
        I = imresize(I, inputSize(1:2));

        C = semanticseg(I, trainedNet);

        maskRGB = zeros(size(C,1), size(C,2), 3);

        for i = 1:numel(selected_classes)
            className = selected_classes(i);
            idxClass = find(classes == className);

            if isempty(idxClass)
                continue;
            end

            classMask = (string(C) == className);
            classColor = master_cmap(idxClass,:);

            for ch = 1:3
                temp = maskRGB(:,:,ch);
                temp(classMask) = classColor(ch);
                maskRGB(:,:,ch) = temp;
            end
        end

        tileImgs_mask{k} = maskRGB;

    catch ME
        warning('Error in image %d: %s', k, ME.message);
        tileImgs_mask{k} = zeros(inputSize(1), inputSize(2), 3);
    end
end

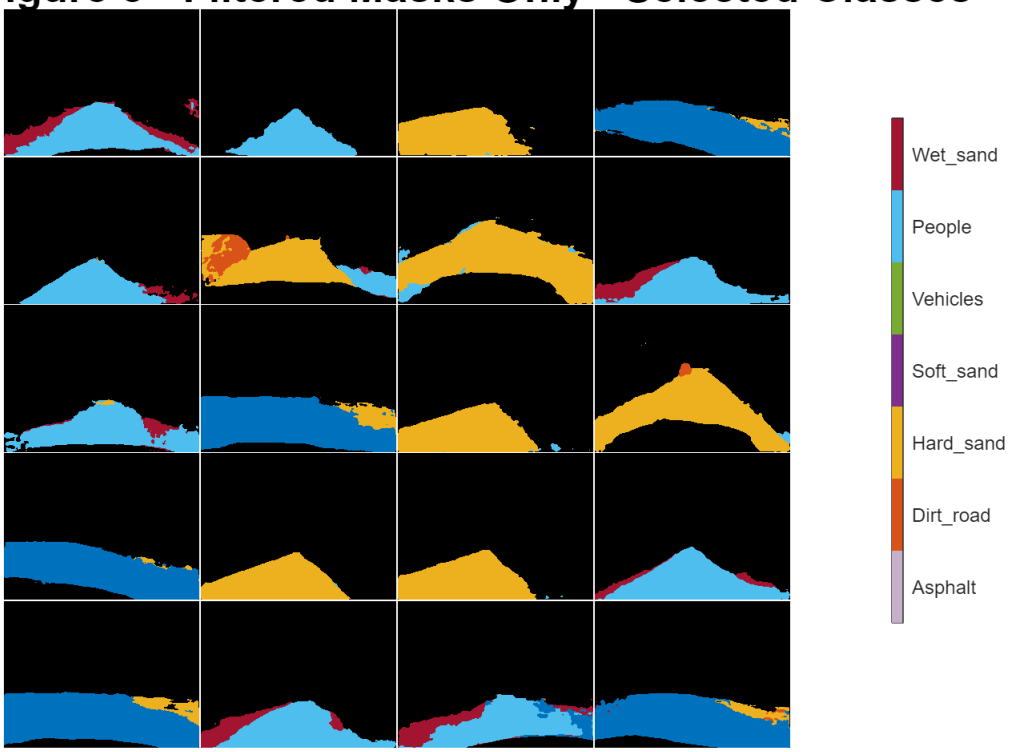
bigImage_mask = imtile(tileImgs_mask, ...
    'GridSize', [5 4], ...
    'BorderSize', 3, ...
    'BackgroundColor', [1 1 1]);

% Main grid
mainAx2 = axes('Position', [0.03 0.05 0.80 0.88]);
imshow(bigImage_mask, 'Parent', mainAx2);
axis(mainAx2, 'off');

% Legend axis (ONLY selected classes)
```

```
legendAx2 = axes('Position', [0.85 0.20 0.03 0.60]);
axis(legendAx2, 'off');
axes(legendAx2);
pixelLabelColorbar_group12(new_master_cmap, selected_classes);
```

Figure 8 - Filtered Masks Only - Selected Classes



4.4 Inspection Of One Image

This code block provides a detailed inspection of a single validation image, allowing for a more in-depth qualitative analysis of the model's performance. A specific image is selected using an index, resized to match the input dimensions of the trained network, and processed through the semantic segmentation model to obtain pixel-wise class predictions. The resulting segmentation is then filtered to retain only a subset of relevant classes, removing all others to focus the analysis. Two visualizations are generated: one showing the full segmentation output and another displaying only the filtered classes. These are presented alongside the original image in a three-panel layout, enabling direct comparison between the input and both levels of prediction. Additionally, a color legend corresponding to the filtered classes is included, ensuring clear interpretation of the segmentation results. This detailed view is particularly useful for examining specific cases, identifying model strengths and weaknesses, and understanding how well the model distinguishes between relevant semantic categories.

```
% Inspect one image in detail
inspectIdx = 6;   % change this to inspect another image

I_raw = imread(testFiles{inspectIdx});
I_resized = imresize(I_raw, inputSize(1:2));
[C_pred, scores] = semanticseg(I_resized, trainedNet);
```

```
C_filtered = C_pred;
maskKeep = ismember(string(C_pred), selected_classes);
C_filtered(~maskKeep) = missing;
C_filtered = removecats(C_filtered);

idx = ismember(classes, categories(C_filtered));
filtered_classes = classes(idx);
filtered_cmap = master_cmap(idx,:);

B_full = labeloverlay(I_resized, C_pred, ...
    'Colormap', master_cmap, ...
    'Transparency', 0.4);

B_filtered = labeloverlay(I_resized, C_filtered, ...
    'Colormap', filtered_cmap, ...
    'Transparency', 0.4);

g12figure("Detailed Inspection of One Image");

% TOP LEFT: Original
ax1 = axes('Position', [0.05 0.55 0.35 0.32]);
imshow(I_resized, 'Parent', ax1);
axis(ax1, 'off');
title(ax1, 'Original Image', 'FontWeight','bold');

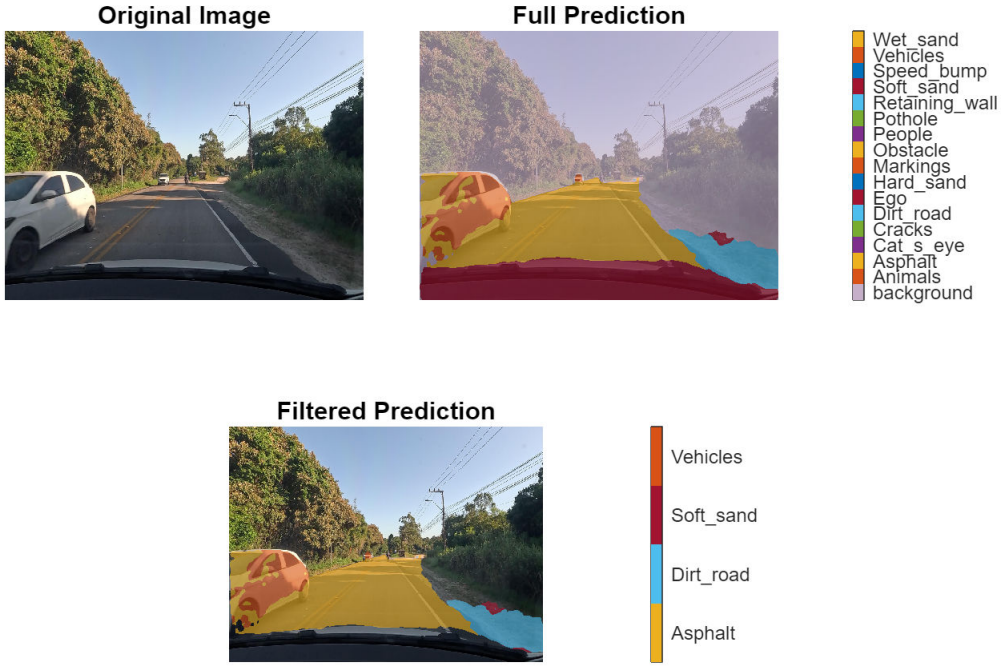
% TOP RIGHT: FULL Prediction
ax2 = axes('Position', [0.42 0.55 0.35 0.32]);
imshow(B_full, 'Parent', ax2);
axis(ax2, 'off');
title(ax2, 'Full Prediction', 'FontWeight','bold');

% Legend (TOP RIGHT)
legendAx1 = axes('Position', [0.80 0.55 0.03 0.32]);
axis(legendAx1, 'off');
axes(legendAx1);
pixelLabelColorbar_group12(master_cmap, classes);

% BOTTOM: Filtered
ax3 = axes('Position', [0.23 0.12 0.35 0.28]);
imshow(B_filtered, 'Parent', ax3);
axis(ax3, 'off');
title(ax3, 'Filtered Prediction', 'FontWeight','bold');

% Legend (BOTTOM)
legendAx2 = axes('Position', [0.62 0.12 0.03 0.28]);
axis(legendAx2, 'off');
axes(legendAx2);
pixelLabelColorbar_group12(filtered_cmap, filtered_classes);
```

Figure 9 - Detailed Inspection of One Image



Note: Depending on the code execution, the image displayed in Figure 9 may vary. Only one result is shown per run.

5. Conclusions

This project successfully built an image segmentation model for autonomous driving in off-road and coastal environments. A DeepLabV3 model that already knew how to recognize city elements (from the CamVid dataset) was adapted using transfer learning to recognize 17 new classes for a specific dataset (FCCD).

Key Achievements:

- Adapting the Network: Successfully modified the final layers of the pre-trained model so it could output 17 categories instead of 11. This allowed the model to shift from understanding standard city roads to understanding complex sand-road environments.
- High Accuracy: The model reached a 93% validation accuracy in just 200 training cycles (epochs) (**Figure 6**). This shows that starting with a pre-trained model was a great choice, as it learned quickly even with a small training set of only 400 images.
- Reliable Visual Results: When testing on new images, the model correctly highlighted important safety obstacles like Vehicles and People, and clearly mapped out different driving surfaces like Asphalt and Sand (**Figure 9**).

Future Work: Even with these great results, the model can still be improved. The next step would be to use data augmentation creating new training examples by slightly tweaking the original dataset images (for example, by

changing the lighting or adding a bit of noise). The goal would be to do this carefully so it wont accidentally ruin the useful visual features the model has already learned.

By safely creating more varied examples, the model will get much better at detecting tricky or rare terrain, like specific types of sand roads and dirt paths. Furthermore, it would improve vehicles detection at various weathers, time of the day, tunnels or even in various sensor conditions, making the autonomous vehicle even safer in the real world.

6. References

[1] Mendeley Data, FCDD: A High-Resolution Unstructured Environment Dataset with Multiple Sand Roads (2025).

[2] MATLAB Documentation: link1, link2.

Appendix. Functions used

```
function fig = g12figure(extraTitle)

    fig = figure;
    clf(fig);

    if nargin == 1 && strlength(extraTitle) > 0
        fullTitle = "Figure " + fig.Number + " - " + extraTitle;
    else
        fullTitle = "Figure " + fig.Number;
    end

    sgtitle(fig, fullTitle, ...
        'FontSize', 16, ...
        'FontWeight', 'bold');

end

function cmap = camvidColorMap_group12()
% Define the colormap used by CamVid dataset.

cmap = [
    128 128 128   % Sky
    64 0 128       % Building
    192 192 192   % Pole
    128 64 128    % Road
    60 40 222     % Pavement
    128 128 0     % Tree
    192 128 128   % SignSymbol
    64 64 128     % Fence
    128 0 0      % Car
    64 64 0       % Pedestrian
    0 128 192     % Bicyclist
```

```
    ];

% Normalize between [0 1].
cmap = cmap ./ 255;
end

function cmap = camvidColorMap_part_2_group12()
% Define the colormap used by CamVid dataset.

cmap = [
    128 128 128   % Sky
    64 0 128       % Building
    192 192 192   % Pole
    128 64 128    % Road
    60 40 222     % Pavement
    128 128 0     % Tree
    192 128 128   % SignSymbol
    64 64 128     % Fence
    128 0 0      % Car
    64 64 0       % Pedestrian
    0 128 192 % Bicyclist
    0 0 0
    0 0 0
    0 0 0
    0 0 0
    0 0 0
    0 0 0
    ];

% Normalize between [0 1].
cmap = cmap ./ 255;
end

function [new_C, new_cmap, new_classes] = g12_modelclass_output(I, C, cmap,
targetNames, allClasses)

    % Find the row numbers of the target names in the full class list
    [~, indices] = ismember(targetNames, allClasses);

    % Automatically build the corresponding C-labels
    targetLabels = "C" + string(indices);

    % Update colormap
    new_cmap = cmap(indices, :);

    % Normalize colormap if necessary
    if max(new_cmap(:)) > 1
        new_cmap = new_cmap ./ 255;
    end
```

```
    % Apply class restriction
    new_C = C;
    restrict = ismember(string(new_C), targetLabels);

    disp('Categories before removal:');
    disp(strjoin(categories(new_C), ', '));

    new_C(~restrict) = missing;

    disp('Pixels matching restricted classes:');
    disp(sum(restrict(:)));

    new_C = removecats(new_C);

    disp('Categories after removal (still C-codes):');
    disp(strjoin(categories(new_C), ', '));

    % Output selected class names
    new_classes = targetNames(:);

end

function pixelLabelColorbar_group12(cmap, classNames)
    % Add a colorbar to the current axis. The colorbar is formatted
    % to display the class names with the color.

    % 1. Ensure cmap is normalized [0, 1] to prevent colormap crashes
    if max(cmap(:)) > 1
        cmap = double(cmap) / 255;
    end

    colormap(gca, cmap);

    % Add colorbar to current figure.
    c = colorbar(gca);

    % 2. DEFINE THE TICKS FIRST
    numClasses = size(cmap, 1);
    c.Ticks = 1/(numClasses*2) : 1/numClasses : 1;

    % 3. ASSIGN THE LABELS SECOND
    c.TickLabels = classNames;

    % 4. Prevent LaTeX underscore formatting errors
    c.TickLabelInterpreter = 'none';

    % Remove tick mark length for a cleaner look.
    c.TickLength = 0;
end
```